Spatial Databases

Storing and querying space at scale — PostGIS, DuckDB, GeoParquet

The question files struggle with

So far we have read a file, held it in memory, and computed. That works beautifully **until the data stops fitting in RAM, or two people need it at once, or the same point-in-polygon question is asked a thousand times a day**. A file has no query engine, no concurrency, no index that survives a restart. A **spatial database** turns "load everything, then filter in Python" into "ask the server a question and get only the rows you need" — and it keeps the geometry, the attributes, and the spatial index together, on disk, for everyone.

> *Roadmap:* this module owns **the relational model, PostGIS, spatial indexing, and joins at scale**. We only *reference* CRS, formats, and GeoDataFrames — those belong to the *Spatial Data* deck.

A file vs. a database

A GeoPackage or GeoParquet file is a wonderful *interchange* format — it is even a SQLite database on disk. But a **server** database adds the things a file cannot: a planner, persistent indexes, transactions, and many readers.

A file (GeoJSON, Shapefile, GPKG)

- You load the **whole** layer, then filter.
- No query planner, no concurrency control.
- Index (if any) rebuilt in memory each run.
- Great for handoff and reproducibility.

A spatial database

- Send **SQL**; get back only matching rows.
- Persistent **spatial index** on disk.
- Transactions + many concurrent users.
- Geometry, attributes & index in one place.

The cross-over point is **scale, sharing, and serving** — the moment data outgrows a laptop or has to back a live application.

What makes a database "spatial"?

A spatial database is an ordinary relational database with **four extensions** bolted onto SQL. PostGIS, SpatiaLite and DuckDB all provide the same quartet — once you recognize it, every engine looks familiar.

1. **Spatial data types** — a `geometry` / `geography` column alongside `int` and `text`, storing points, lines and polygons.
2. **Spatial functions** — hundreds of `ST_*` functions to measure, construct, transform and relate geometries.
3. **Spatial relationships** — boolean predicates (*contains*, *intersects*, *within*) standardized by the **DE-9IM** model.
4. **Spatial indexing** — an R-tree-style index so a query touches a handful of candidate rows instead of scanning the whole table.

Everything else in this deck is just these four ideas in practice.

The landscape: three tiers

There is no single "best" store — pick the tier that matches the job. The same indexing and join ideas live in all three.

File-embedded

GeoPackage, SpatiaLite

- Single file, zero server.
- Interchange + small apps.
- (Deck 02's default format.)

Server (OLTP)

PostgreSQL + PostGIS

- Multi-user, transactional.
- Serving live applications.
- The reference implementation.

Analytical (OLAP): DuckDB + GeoParquet — in-process, columnar, built to scan huge datasets *and to query files in place*. We close the deck here.

One-liners worth knowing: **ibis** = one Python API over either engine; **H3** = hexagonal global grid index; **SpatiaLite** = PostGIS-like power inside a SQLite file.

OGC Simple Features: WKT & WKB

Before any engine, there is a **standard**. The OGC *Simple Features for SQL* specification defines the geometry types and two interchange encodings every tool agrees on — so a polygon written by PostGIS reads cleanly in DuckDB.

WKT — Well-Known *Text*, human-readable:

```
'POINT(11.34 44.49)'
'LINESTRING(11.34 44.49, 11.35 44.50)'
'POLYGON((0 0, 4 0, 4 4, 0 4, 0 0))'
```

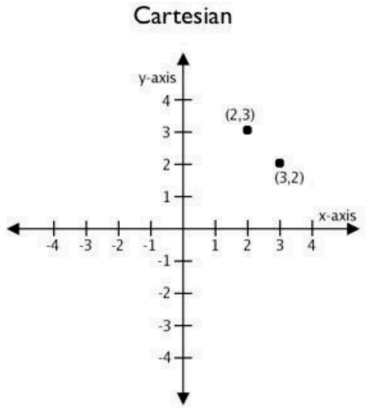
WKB — Well-Known *Binary*: the same geometry, compact and exact. It is what columns actually store.

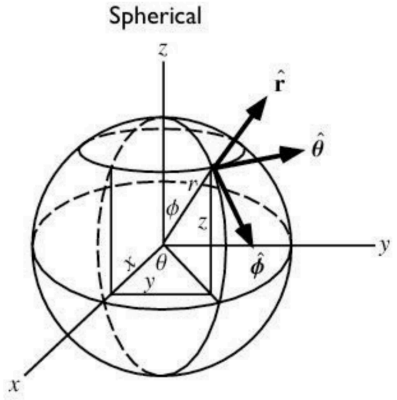
```
-- text  <->  geometry  <->  binary
SELECT ST_GeomFromText('POINT(11.34 44.49)', 4326);  -- parse + SRID
SELECT ST_AsText(geom);      -- geometry back to WKT
SELECT ST_AsBinary(geom);    -- geometry to WKB
```

The number `4326` is the **SRID** — the CRS, stored *with* the data.

Geometry vs. geography

PostGIS offers **two** spatial types. **geometry** treats coordinates as points on a **flat plane** — fast, with the full function set. **geography** treats them as **lon/lat on the spheroid** and measures along great circles — slower, fewer functions, but correct over long distances and across the antimeridian.






```
geometry(Point, 4326)   -- planar; fast; many ops
geography(Point, 4326)  -- geodetic; metres; fewer ops
```

Rule of thumb: store as **geometry** and reproject to a metric CRS for measurement; reach for **geography** only for global lon/lat distances. *DuckDB is planar-only — it has no **geography** type.*

Create & load a spatial table

The SRID is not a separate column — it is a **parameter of the type**, `geometry(Point, 4326)`. Enable the extension once, declare the typed column, then build a **GiST** index so queries can use it.

```
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE photos (
  pid  int PRIMARY KEY,
  geom geometry(Point, 4326)
);

CREATE INDEX photos_gix
  ON photos
  USING GIST (geom);
```

Getting data in:

- `shp2pgsql -s 32633 r.shp r | psql gis`
- `ogr2ogr -f PostgreSQL PG:dbname=gis x.geojson`
- `INSERT … ST_GeomFromText(wkt, 4326)`
- `ST_GeomFromGeoJSON(…)`
- `\copy photos FROM 'p.csv' CSV HEADER`

Reproduce a server in one line: `docker run --rm -e POSTGRES_PASSWORD=pw -p 5432:5432 postgis/postgis:16-3.6`, then `CREATE EXTENSION postgis;`. (PostGIS 3.6.2, Feb 2026.)

Spatial functions: one reference slide

Hundreds of `ST_*` functions, but they fall into a handful of families. Learn the families, not the list — and remember measurement needs a **metric CRS**.

- Construct: `ST_MakePoint`, `ST_GeomFromText`, `ST_GeomFromGeoJSON`
- Inspect: `ST_SRID`, `ST_X`, `ST_GeometryType`, `ST_NPoints`
- Measure: `ST_Area`, `ST_Length`, `ST_Distance`, `ST_Perimeter`
- Validity: `ST_IsValid`, `ST_MakeValid`
- Process: `ST_Buffer`, `ST_Centroid`, `ST_Union`, `ST_Intersection`, `ST_Simplify`
- Relate: `ST_Contains`, `ST_Within`, `ST_Intersects`, `ST_DWithin`
- Reproject: `ST_Transform`, `ST_SetSRID`
- Output: `ST_AsText`, `ST_AsGeoJSON`, `ST_AsMVT`

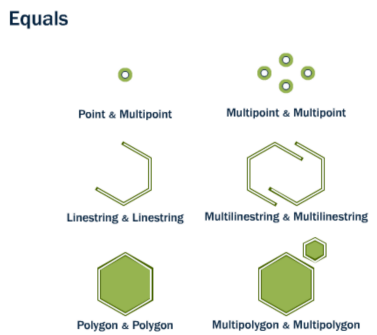
```
-- measure in metres: reproject first, then ST_Area
SELECT name, ST_Area(ST_Transform(geom, 32633)) AS m2 FROM parks;
```

The relationships model: DE-9IM

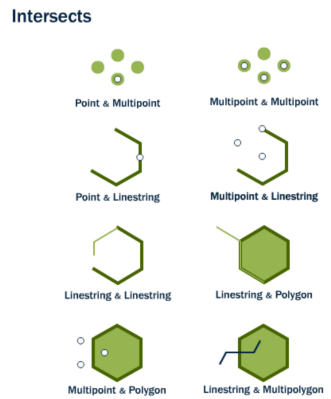
Predicates like *contains* and *touches* are not ad-hoc. Every geometry is split into **interior, boundary, exterior**; the **DE-9IM** matrix records how two geometries' parts intersect, and every named predicate is just a pattern over that 3×3 matrix. This is the OGC standard PostGIS, GEOS and DuckDB share.

- **Interior** — inside, off the boundary.
- **Boundary** — the separating edge.
- **Exterior** — everything else.

A predicate = a required intersection pattern between these parts of A and B. **ST_Relate** exposes the raw matrix.



ST_Equals



ST_Intersects

A gallery of predicates

The same matrix yields the predicates you reach for daily. Each is one boolean **`ST_*`** call; the picture is worth more than the definition.

ST_Overlaps

```
ST_Equals(a, b)       -- same set of points
ST_Intersects(a, b)   -- share any point (not disjoint)
ST_Contains(a, b)     -- b lies fully inside a
ST_Within(a, b)       -- a lies fully inside b
```

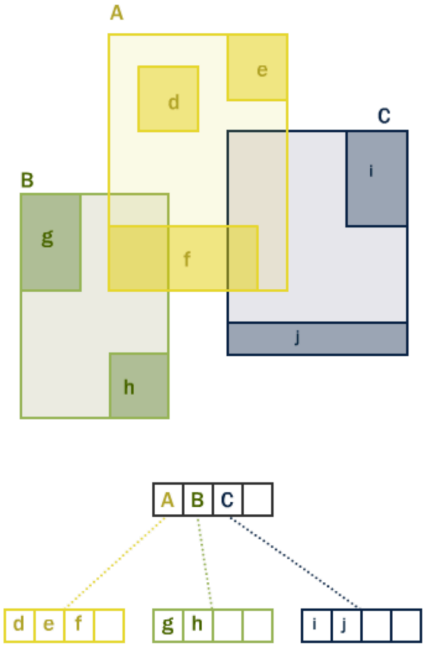
ST_Touches

```
ST_Touches(a, b)      -- meet only on boundaries
ST_Overlaps(a, b)     -- partial, same dimension
ST_Crosses(a, b)      -- partial, lower dimension
ST_DWithin(a, b, d)   -- within distance d
```

Spatial indexing: GiST & the R-tree

This is the heart of the module. Without an index, a predicate compares every row against every other — **O(N×M)**. The fix is an **R-tree**: a hierarchy of nested **bounding boxes** that lets a query discard whole branches at once. Boxes nest into parents (A, B, C); leaves hold the real geometries (d…j). A search descends only the boxes its query box touches.
In PostGIS the implementation is **GiST** (`USING GIST`); the R-tree is the *concept*, GiST the *mechanism*. DuckDB writes it as `USING RTREE`.

Same idea as Deck 02's GeoDataFrame `.sindex` — here it is pushed into the database and persisted.

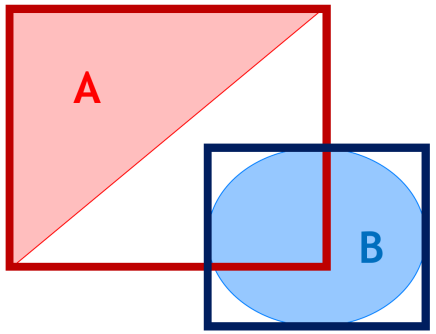


Two-pass: filter, then refine

A bounding box is a *fast lie*: boxes can overlap while the true shapes do not. So spatial queries run in **two passes** — the index (`&&`) cheaply returns *candidates* whose boxes overlap, then the exact topological test (`ST_Intersects`) confirms the real answer on that short list.

Here A's box and B's box overlap, so `A && B = TRUE` — yet the triangle and circle never meet, so the exact test is `FALSE`.

Only **bounding-box operators** (`&&`) use the index. The good news: functions like `ST_Intersects` and `ST_DWithin` call `&&` internally — so you write the exact predicate and get the index for free.



A && B = **TRUE**

_ST_Intersects(A && B) = **FALSE**

* _ST_Intersects: geometric intersection

Writing index-aware queries

Because the planner needs a bounding-box operator to reach the GiST index, the shape of the query matters. The reliable pattern is **ST_DWithin** (not **ST_Distance(…) < d**) and keeping the column geometry untouched on its side of the predicate.

```
-- "within 500 m of a point": ST_DWithin uses the index;
--  ST_Distance(geom, p) < 500 would NOT.
SELECT name
FROM   roads                          -- geom stored in EPSG:32633 (metres)
WHERE  ST_DWithin(
         geom,
         ST_Transform(
           ST_SetSRID(ST_MakePoint(11.34, 44.49), 4326), 32633),
         500);                        -- 500 metres, because geom is metric
```

Wrap the *constant* in **ST_Transform**, never the indexed column — transforming **geom** would defeat the index on every row.

The spatial JOIN — the climax

Everything converges here. A **spatial join** matches rows from two tables by a *geometric* predicate instead of a key — and the GiST index is what makes it scale from a toy to millions of rows.

```sql
-- Point-in-polygon: how many listings fall in each county?
SELECT c.name, count(*) AS n
FROM   counties c
JOIN   listings  p  ON ST_Contains(c.geom, p.geom)
GROUP  BY c.name
ORDER  BY n DESC;
```

```sql
-- Proximity join: every hospital within 2 km of each school
SELECT s.id, h.id
FROM   schools   s
JOIN   hospitals h  ON ST_DWithin(s.geom, h.geom, 2000);  -- metres
```

ST_Contains for *inside*, **ST_DWithin** for *near*: two predicates cover most spatial-join work you will ever write.

PostGIS from Python — the bridge

You rarely choose *either* PostGIS *or* GeoPandas — you move between them. GeoPandas speaks PostGIS directly, so the database does the heavy spatial filter and pandas does the rest.

```python
import geopandas as gpd
from sqlalchemy import create_engine

engine = create_engine("postgresql://user:pass@localhost/gis")

# DB -> GeoDataFrame (let PostGIS do the spatial work first)
gdf = gpd.read_postgis(
    "SELECT name, geom FROM counties", engine, geom_col="geom")

# GeoDataFrame -> DB
gdf.to_postgis("counties", engine, if_exists="replace")
```

The same bridge to the analytical world is a **GeoParquet** file: GeoPandas and DuckDB both read and write it, so it is their lingua franca — next.

Pivot: from serving to analytics

PostGIS is a **transactional server** — superb at serving a live map and guarding writes from many users. But "scan 200 million points and aggregate" is a different job: **analytical (OLAP)**, columnar, single-analyst, often read-only.
That is **DuckDB**: an in-process engine (like SQLite, but columnar and vectorized) with a **spatial** extension. Same SQL, same **ST_*** predicates, same R-tree idea — but built to chew through files.

- The **GEOMETRY type is core** since DuckDB v1.5.0 (2026); the **spatial** extension adds the **ST_*** functions, **ST_Read**, and GeoParquet I/O.
- No server, no **CREATE EXTENSION** ceremony — **pip install duckdb** and go.
- Its punchline, two slides on: it can **query files in place**, so the *load* step often disappears entirely.

DuckDB hands-on (1): connect & build tables

A real session — **pip install duckdb**, no server. The **GEOMETRY** type is core; we **LOAD spatial** only for the **ST_*** functions and **ST_Read**. We rebuild exactly Deck 02's Texas example: counties (polygons) × Airbnb listings (points).

```
import duckdb
con = duckdb.connect()                          # in-process, no server
con.execute("INSTALL spatial; LOAD spatial;")  # ST_* + ST_Read
```

```
-- polygons straight from a GeoJSON file via ST_Read
CREATE TABLE counties AS
  SELECT NAME AS county, geom FROM ST_Read('texas.geojson');

-- points built from lon/lat columns of a CSV
CREATE TABLE listings AS
  SELECT id, ST_Point(longitude, latitude) AS geom
  FROM read_csv_auto('listings.csv.gz');
-- counties: 254   listings: 5835
```

DuckDB hands-on (2): the spatial join

The same point-in-polygon join as PostGIS, on the same data — and it returns **exactly Deck 02's GeoPandas result** (Travis = Austin, 5792 listings). One analytical engine, one SQL query, zero server.

```
SELECT c.county, count(*) AS n
FROM   counties c
JOIN   listings p ON ST_Contains(c.geom, p.geom)
GROUP  BY c.county
ORDER  BY n DESC
LIMIT  3;
```

```
┌──────────────┬──────┐
│   county     │   n  │
├──────────────┼──────┤
│ Travis       │ 5792 │
│ Williamson   │   35 │
│ Hays         │    7 │
└──────────────┴──────┘
-- 5834 of 5835 points matched a county (one stray geocode falls outside).
```

DuckDB hands-on (3): index & GeoParquet

Two more real outputs. DuckDB's index is **USING RTREE** (same concept, PostGIS calls it GiST). And a **GeoParquet** round-trip: the columnar file format that is the interchange currency between DuckDB and GeoPandas.

```
CREATE INDEX idx ON listings USING RTREE (geom);   -- R-tree, on disk
SELECT count(*) FROM listings
WHERE ST_DWithin(geom, ST_Point(-97.7431, 30.2672), 0.05);
-- -> 3938 listings near central Austin
```

```
-- write GeoParquet, then read it straight back
COPY (SELECT id, geom FROM listings) TO 'listings.parquet';
SELECT count(*), ST_GeometryType(geom) FROM 'listings.parquet'
GROUP BY 2;                          -- -> 5835 | POINT
-- file carries a 'geo' metadata key -> GeoPandas reads it as a GeoDataFrame
```

Query files in place

The modern punchline: with **httpfs**, DuckDB queries **remote GeoParquet without downloading it** — only the bytes the query needs travel. The *load* step becomes optional. This is how **Overture Maps** is meant to be used.

```
INSTALL httpfs; LOAD httpfs;          -- read over HTTPS / S3

-- count features straight out of a remote GeoParquet (executed):
SELECT count(*), ST_GeometryType(geometry) AS g
FROM 'https://.../geoparquet/examples/example.parquet'
GROUP BY 2;
-- -> 3 MULTIPOLYGON, 2 POLYGON  (read in place, nothing saved locally)
```

```
# real Overture pattern (S3); keep a local file as offline fallback
src = "s3://overturemaps-us-west-2/release/2026-.../*.parquet"
# con.execute(f"SELECT ... FROM read_parquet('{src}') WHERE bbox...")
```

GeoParquet is **mid-transition**: 1.1 (WKB + metadata, what tools emit today) → native Parquet geometry → GeoParquet 2.0.

PostGIS vs. DuckDB: when to use which

Not rivals — **two jobs**. Use the transactional server to *serve*; use the analytical engine to *crunch*. GeoParquet is the bridge between them.

Reach for PostGIS when…

- Many users read *and write* concurrently.
- You serve a live app / web map.
- You need transactions & constraints.
- You need geodetic `geography`.

Reach for DuckDB when…

- One analyst, big read-only scans.
- Data already lives in Parquet / on S3.
- You want zero-setup, in-process SQL.
- You want to query files in place.

Same SQL, same `ST_*`, same R-tree. Learn the predicates once; switch engines as the job demands — or use `ibis` to write one query for either.

Summary & further learning

- A **spatial database** = SQL + four extensions: **types, functions, predicates (DE-9IM), and indexing**.
- **PostGIS** is the transactional reference; `geometry` (planar) vs `geography` (geodetic); SRID is a **type parameter**, not a column.
- **GiST / R-tree** + the **two-pass** filter-and-refine make spatial **joins** scale — the whole point of a spatial DB.
- **DuckDB + GeoParquet** is the analytical tier: in-process, columnar, and able to **query files in place**.

PostGIS manual & "Intro to PostGIS" workshop — https://postgis.net/documentation/ · https://postgis.net/workshops/postgis-intro/ | DuckDB spatial — https://duckdb.org/docs/stable/core_extensions/spatial/overview | GeoParquet — https://geoparquet.org/ · Overture Maps — https://overturemaps.org/ | ibis — https://ibis-project.org/ · H3 — https://h3geo.org/